Computer Organization and Assembly Language

Week 10 — Chapter 4: Bit Manipulations
Logical Operations, Masking and Bit Testing

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 10

Outline

1. Quick Review
2. Why This Topic in the Chapter?
3. AND, OR, XOR, NOT
4. Masking Operations
5. Example 4.3

Quick Review: What Was Done Last Week?

- **Shifts:** `SHR` (unsigned $\div 2$), `SHL`/`SAL` ($\times 2$), `SAR` (signed $\div 2$ — copies sign bit).
- **Rotations:** `ROR`, `ROL` lose nothing; `RCR`, `RCL` put carry inside the ring (9-bit rotation).
- **Extended shift left:** `SHL [low], 1` then `RCL [high], 1` — CF bridges the two halves.
- **Extended shift right:** `SHR [high], 1` then `RCR [low], 1` — high half goes first.
- **Extended addition:** `ADD` low halves, then `ADC` high halves — carry propagates automatically.
- **Multiply loop:** shift multiplier right $\rightarrow$ if CF=1 add multiplicand $\rightarrow$ shift multiplicand left $\rightarrow$ repeat 16 times.
- A 16-bit number shifted left must live in 32-bit space (`dd`) or significant bits fall off.

Why Are We Studying Logical Operations?

The question that opens this section

We can shift bits and do extended arithmetic. But **how do we inspect, set, clear, or flip individual bits** inside a byte or word?

- **Logical operations are bitwise.** Every bit is processed independently and in parallel — 16 ANDs at once for a 16-bit operand.
- **AND clears bits.** Force selected bits to 0.
- **OR sets bits.** Force selected bits to 1.
- **XOR flips bits — and is reversible.** $(A \oplus B) \oplus B = A$. Foundation of cryptography.
- **NOT inverts everything.** Use XOR for selective inversion.

The real lesson

Assembly gives you only basic building blocks. What you build from them is limited by your **imagination**, not the instruction set.

AND Operation

- **AND** performs bitwise AND. A result bit is **1 only if BOTH** corresponding bits are 1.

X	Y	X and Y
0	0	0
0	1	0
1	0	0
1	1	1

Bitwise example — `and ax, bx`

1	0	1	1	0	1	0	0
1	1	1	1	0	0	0	0
1	0	1	1	0	0	0	0

Upper nibble kept (mask=1) Lower nibble cleared (mask=0)

```
and ax, bx              ; AX = AX AND BX
and byte [mem], 5       ; memory AND immediate
and al, 0F0h            ; clear lower nibble of AL
```

Key point

AND is used to **CLEAR** bits. Mask bit = 1 → keep. Mask bit = 0 → clear to 0.

OR Operation

- **OR** performs bitwise inclusive OR. A result bit is **1 if EITHER or BOTH** corresponding bits are 1.

X	Y	X or Y
0	0	0
0	1	1
1	0	1
1	1	1

Bitwise example — `or ax, bx`

1	0	1	0	0	0	0	0
0	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1

Upper nibble kept (mask=0) Lower nibble set to 1 (mask=1)

```
or  ax, bx              ; AX = AX OR BX
or  byte [mem], 5       ; set bits 0 and 2 of memory byte
or  al, 0Fh             ; force lower nibble of AL to all 1
```

Key point

OR is used to **SET** bits. Mask bit = 1 → force to 1. Mask bit = 0 → keep original.

XOR Operation (Exclusive OR)

- **XOR** produces 1 only where bits **DIFFER**. Same bits → 0. **Reversible:** $(A \oplus B) \oplus B = A$.

X	Y	X xor Y
0	0	0
0	1	1
1	0	1
1	1	0

Bitwise example — `xor ax, bx`

1	0	1	1	0	1	0	0
0	0	0	0	1	1	1	1
1	0	1	1	1	0	1	1

Upper nibble unchanged (mask=0) Lower nibble flipped (mask=1)

```
xor ax, bx          ; flip bits wherever BX is 1
xor ax, ax          ; zero AX instantly (faster than mov
xor byte [mem], 5;  invert bits 0 and 2 of memory byte
```

Why XOR is special

Reversible — used in cryptography, image drawing, checksums. `xor ax, ax` is the canonical way to zero a register.

NOT Operation

- **NOT** inverts every bit (one's complement). **Single operand** — no source, no mask.

X	NOT X
0	1
1	0

Bitwise example — `not al`

1	0	1	1	0	1	0	0
0	1	0	0	1	0	1	1

```
not ax                  ; invert all 16 bits of AX
not byte [mem]          ; invert all 8 bits of memory byte
; NOT has only ONE operand — no immediate form exists
```

NOT vs XOR — Remember

NOT inverts *all* bits — no choice. **XOR with mask** inverts only selected bits (mask bit=1 flips).
NOT does **not** change flags — **cannot use `JZ` or `JC` after NOT.**
AND/OR/XOR always clear CF and OF.

All Four Logical Instructions at a Glance

Instr.	Result = 1 when...	Common use	Flags
`AND`	Both bits = 1	Clear bits (masking)	CF=0, OF=0; ZF/SF/PF set
`OR`	Either bit = 1	Set bits selectively	CF=0, OF=0; ZF/SF/PF set
`XOR`	Bits differ	Flip bits; zero register; encrypt	CF=0, OF=0; ZF/SF/PF set
`NOT`	Always (all bits)	One's complement	**No flags changed**

Flags rule

AND, OR, XOR always **clear CF and OF**. After these, `JC` and `JO` never fire — only `JZ`/`JNZ` and `JS` are useful.

- **Operand forms** (same as ADD): `reg,reg` `reg,mem` `reg,imm` `mem,imm` — NOT: single operand only.

Masking Operations

What is a mask?

A **mask** is a pattern of bits used as a stencil to act on selected positions while leaving the rest untouched.

Operation	Mask 1-bit	Mask 0-bit	Example
AND — Clear	keep original	**force to 0**	`AND F0h`
OR — Set	**force to 1**	keep original	`OR 0Fh`
XOR — Invert	**flip**	keep original	`XOR 0Fh`
TEST — Check	**test position**	ignore	`TEST 01h`

The universal mask rule

1-bits in mask = positions to act on.
0-bits in mask = leave unchanged (AND/XOR) or keep (OR).

Selective Bit Clearing — AND

- Use AND: **0** in mask at positions to clear, **1** to keep.

AND masking rule

Mask bit $= 1 \Rightarrow$ destination bit **KEPT** $(x \text{ AND } 1 = x)$

Mask bit $= 0 \Rightarrow$ destination bit **CLEARED** $(x \text{ AND } 0 = 0)$

Example — clear lower nibble using mask `F0h`

AL =	1	0	1	1	1	1	0	1
Mask F0h =	1	1	1	1	0	0	0	0
Result =	1	0	1	1	0	0	0	0

upper nibble unchanged | lower nibble cleared

```
and al, 0F0h          ; clear lower nibble
and ax, 0FF00h        ; clear lower byte of AX
and byte [mem], 0FEh  ; clear bit 0 of a memory byte
```

Selective Bit Setting — OR

- Use OR: **1** in mask at positions to set, **0** to keep.

OR masking rule

Mask bit $= 1 \Rightarrow$ destination bit **SET to 1** $(x \text{ OR } 1 = 1)$

Mask bit $= 0 \Rightarrow$ destination bit **KEPT** $(x \text{ OR } 0 = x)$

Example — force lower nibble to all 1s using mask `0Fh`

AL =	1	0	1	1	0	0	1	0
Mask 0Fh =	0	0	0	0	1	1	1	1
Result =	1	0	1	1	1	1	1	1

upper nibble unchanged | lower nibble set to 1

```
or al, 0Fh          ; set lower nibble
or ax, 0001h        ; set bit 0 of AX
or byte [mem], 80h  ; set the MSB of a memory byte
```

Selective Bit Inversion — XOR

- Use XOR: **1** in mask at positions to flip, **0** to keep.

XOR masking rule

Mask bit $= 1 \Rightarrow$ destination bit **FLIPPED** $(x \text{ XOR } 1 = \overline{x})$

Mask bit $= 0 \Rightarrow$ destination bit **KEPT** $(x \text{ XOR } 0 = x)$

Example — flip lower nibble using mask `0Fh`

AL =	1	0	1	1	0	1	0	0
Mask 0Fh =	0	0	0	0	1	1	1	1
Result =	1	0	1	1	1	0	1	1

upper nibble unchanged lower nibble flipped

```
xor al, 0Fh      ; flip lower nibble
xor ax, 0FFFFh   ; same effect as NOT ax
xor al, 80h      ; flip sign bit only
```

XOR vs NOT

NOT flips *every* bit. XOR flips *only* where mask = 1 — selective inversion.

Selective Bit Testing — AND and TEST

New approach — more powerful than shifting

Previously: `SHR` pushed bits into CF one at a time, in order.
Now: AND with a 1 at the desired position. If result = 0 (ZF=1), the bit was 0.
Any bit, any time, any order.

```
and al, 04h          ; mask = 0000 0100b (isolate bit 2)
jz  bit2_clear       ; ZF=1 -> result was 0 -> bit 2 was 0
; if we reach here, bit 2 was 1
```

Problem with AND for testing

AND writes its result into the destination — the **original value is destroyed**. If you need it again, save it first.

Solution: the `TEST` instruction — a non-destructive AND that only sets flags.

The TEST Instruction — Non-Destructive AND

TEST — analogous to CMP

`CMP` = non-destructive SUB; `TEST` = non-destructive AND. Both only set flags — neither operand changes.

`test ax, bx` ⇒ flags set as if `and ax, bx` was done.

	CMP (magnitude)	**TEST (bit check)**
Operation	Subtraction internally	AND internally
Result written?	No — flags only	No — flags only
Operands?	Neither changed	Neither changed
Jump after	JE/JNE/JL...	JZ/JNZ
Checks	Is A = / < / > B?	Is a specific bit set?

```
test al, 01h ; is bit 0 set? AL is NOT changed
jz even ; ZF=1 -> bit was 0 (even); ZF=0 -> odd
```

Remember — TEST always clears CF

Use JZ/JNZ after TEST, never JC. Both operands are unchanged after TEST.

Example 4.3 — 16-bit Multiplication Using TEST

```
; 16-bit multiplication using test for bit testing
[org 0x0100]
            jmp start

multiplicand: dd 1300 ; 16-bit multiplicand in 32-bit space
multiplier: dw 500 ; 16-bit multiplier
result: dd 0 ; 32-bit result

start: mov cl, 16 ; initialize bit count to 16
         mov bx, 1 ; initialize bit mask to 0001h

checkbit: test bx, [multiplier] ; test current bit -- nothing changes
         jz skip ; skip addition if bit is zero
         mov ax, [multiplicand]
         add [result], ax ; add less significant word
         mov ax, [multiplicand+2]
         adc [result+2], ax ; add more significant word

skip: shl word [multiplicand], 1
         rcl word [multiplicand+2], 1 ; shift multiplicand left (32-bit)
         shl bx, 1 ; shift mask left to test next bit
         dec cl ; decrement bit count
         jnz checkbit ; repeat if bits left

         mov ax, 0x4c00
         int 0x21
```

- **BX** holds the mask. `TEST` is non-destructive — `[multiplier]` is never modified.

Example 4.3 — What Changed and Why

	Example 4.2 (SHR)	**Example 4.3 (TEST)**
Bit testing	`shr dx, 1` then `jnc skip` — shifts multiplier, reads CF	`test bx, [multiplier]` then `jz skip` — mask in BX; multiplier unchanged
Mask	Not needed — SHR exposes bits by shifting	BX = 1, then `shl bx,1` each loop moves 1-bit left
Multiplier	DX — shifted right each loop	`[multiplier]` in memory — never changed
Optimization	Must always count 16 iterations	Can stop when BX = 0 (all bits tested)
Result	`0009EB10h` = 650,000	`0009EB10h` = 650,000 ✓

Debugger observation & clever trick

`[multiplier]` never changes (non-destructive TEST). BX shifts left: `0001` → `0002` → `0004` → ... Replace `dec cl / jnz` with `test bx, bx / jnz` — when BX = 0, loop ends automatically.

Example 4.3 — Traced, 1300 × 500

500 = 0x01F4 = 0000 0001 1111 0100$_2$ Mask BX starts at 0001h (bit 0 first)

Iter	BX (mask)	TEST result	Add?	Result (hex)
1	0001h	0 (bit 0=0)	skip	0000 0000
2	0002h	0 (bit 1=0)	skip	0000 0000
3	0004h	≠0 (bit 2=1)	+1300	0000 0514
4	0008h	0 (bit 3=0)	skip	0000 0514
5	0010h	≠0 (bit 4=1)	+2600	0000 0B18
...(16 iterations total)...				

- [multiplier] does **NOT** change — TEST is non-destructive.
- BX (mask) shifts left: 0001 → 0002 → 0004 → ...
- multiplicand doubles each loop via SHL + RCL (32-bit shift).

Final result after all 16 iterations

result = 0009EB10h = 650,000 (1300 × 500 = 650,000) ✓

Summary

- **AND:** both bits $= 1 \rightarrow$ result 1. Clears CF and OF.
- **OR:** either bit $= 1 \rightarrow$ result 1. Clears CF and OF.
- **XOR:** bits differ $\rightarrow$ result 1. Reversible: $(A \oplus B) \oplus B = A$. Clears CF and OF.
- **NOT:** inverts all bits. Single operand. Does **not** change flags.
- **AND masking:** 0 in mask $\rightarrow$ clears bit. 1 in mask $\rightarrow$ keeps original.
- **OR masking:** 1 in mask $\rightarrow$ sets bit. 0 in mask $\rightarrow$ keeps original.
- **XOR masking:** 1 in mask $\rightarrow$ flips bit. 0 in mask $\rightarrow$ keeps original.
- **TEST:** non-destructive AND — sets flags only, changes nothing. Use `JZ`/`JNZ` after.
- **Example 4.3:** BX holds the mask; `shl bx, 1` each loop tests the next bit without touching `[multiplier]`.

Thank You

Any Questions?

"Assembly language only gives you the basic building blocks. Whether you build a hut or a palace from them depends only on your imagination."

Next: Chapter 5 — Subroutines

What is a subroutine? CALL and RET Passing parameters Nested calls